// DEEP DIVE

Digital Forensics Basics

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

Preserving evidence in the right order, imaging a disk without altering it, reading Windows artifacts, and building a timeline that actually holds up — from first response to final report.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	Core principles, and the order of volatility during acquisition
3 — 4	Memory forensics, and disk forensics/imaging
5 — 7	Chain of custody, timeline analysis, and Windows artifacts
8 — 9	A worked forensic investigation, and reference

How to use this guide: read start to finish for the full picture, or jump to Section 8 for the worked investigation walkthrough. Every diagram is numbered and referenced directly in the surrounding text.

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

What Digital Forensics Is & Core Principles

1.1 BEYOND "WHAT HAPPENED" — PROVABLY WHAT HAPPENED

Digital forensics is the scientific process of identifying, preserving, analyzing, and documenting digital evidence in a way that's defensible — able to withstand scrutiny, whether that's an internal review, a regulator, or a courtroom. This is a meaningfully higher bar than the triage work covered in the Linux for SOC Analysts and Windows Event Log Reference guides: triage answers "what's going on right now," forensics answers "what happened, provably, in a way that will hold up later."

1.2 THE THREE CORE PRINCIPLES

PRINCIPLE	MEANS
Preservation	The original evidence is never modified — all analysis happens on a copy (Section 4.2), never the source
Documentation	Every action taken, by whom, and when, is recorded — the chain of custody (Section 5)
Reproducibility	Another qualified analyst, given the same evidence and methods, should reach the same conclusions

> **"Don't touch the original" is the single most important rule in this entire guide.** Nearly every technique in the sections that follow — write-blocking (Section 4.2), hashing before and after acquisition (Section 4.3), careful order-of-volatility collection (Section 2) — exists specifically in service of this one principle. Modify the original evidence, even accidentally, and its value as evidence can be permanently compromised.

1.3 WHERE THIS FITS RELATIVE TO INCIDENT RESPONSE

Forensics and incident response overlap heavily but aren't identical — the IR Playbook Companion guide's Containment phase (Section 4) explicitly flagged the tension between acting fast and preserving evidence; this guide is the detailed answer to "how do you actually preserve it correctly" once that decision is made. In practice, forensic collection often happens *during* IR's containment and identification phases, not as a separate activity afterward.

// SECTION 2

Order of Volatility & Evidence Acquisition

2.1 SOME EVIDENCE DISAPPEARS FASTER THAN OTHERS

Different types of digital evidence persist for very different lengths of time — RAM contents vanish the moment a system loses power, while data on disk can persist for years. The **order of volatility** is the standard collection sequence, most volatile first, ensuring the evidence most likely to be lost is captured before anything that can afford to wait.

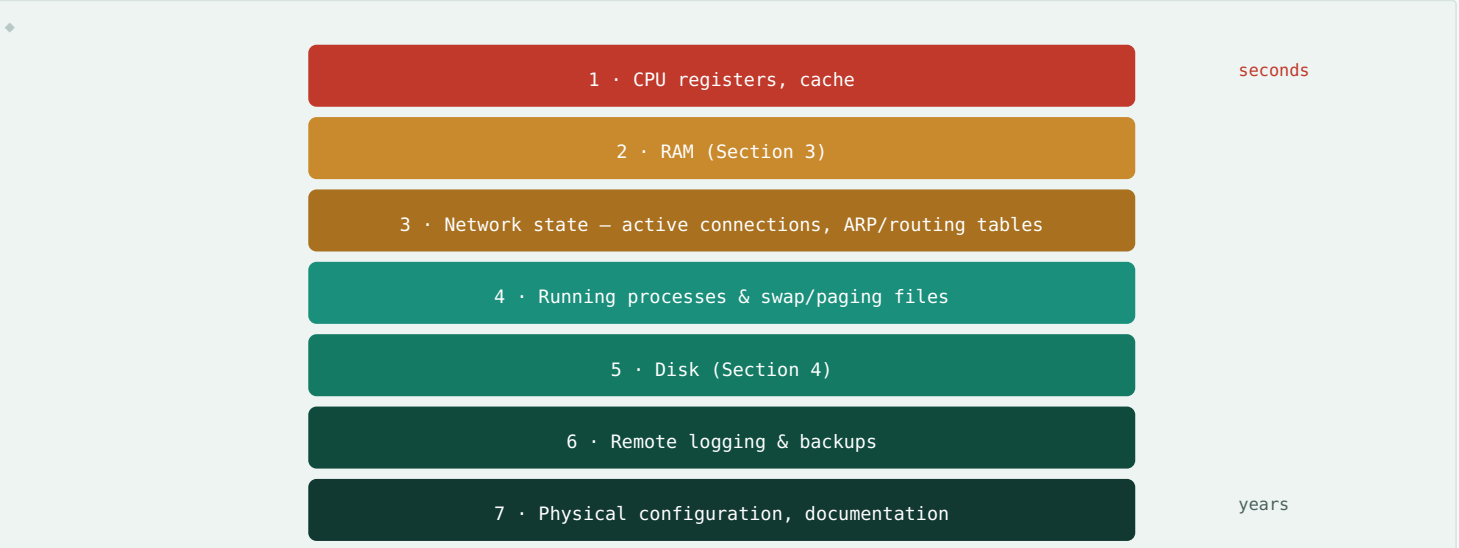

Fig 2.1 — The order of volatility: collect from the top down — what disappears in seconds first, what persists for years last.

2.2 WHY THIS ORDER ISN'T ARBITRARY

Powering off a system to "safely" image the disk (Section 4) destroys everything above it in Fig 2.1 — running processes, active network connections, and RAM contents (Section 3) are gone permanently the instant power is lost. This is precisely the same trade-off flagged in the IR Playbook Companion guide's Section 4.3: acting to contain a threat by isolating or powering off a host can destroy exactly the volatile evidence that would have explained what actually happened.

2.3 THE PRACTICAL SEQUENCE

In practice, this means: capture memory (Section 3) and active network state *before* isolating a host from the network, and image the disk (Section 4) only after volatile evidence is already safely captured — not the instinctive "unplug it immediately" reaction that live incident pressure often produces.

// SECTION 3

Memory Forensics

3.1 WHAT ONLY LIVES IN RAM

A memory (RAM) capture preserves evidence that simply doesn't exist anywhere on disk: running processes and their full command lines, network connections active at the moment of capture, encryption keys held in memory, and malware that runs entirely in-memory without ever writing itself to disk — a technique specifically designed to evade the disk-based static analysis covered in the Malware Analysis Basics guide, Section 3.

3.2 CAPTURING MEMORY WITHOUT DESTROYING IT

Tools like `FTK Imager`, `WinPmem`, or `LiME` (Linux) capture the full contents of RAM to a file, ideally written to external storage rather than the suspect system's own disk (which would overwrite disk-based evidence, Section 4, in the process). The capture itself necessarily uses a small amount of the very memory it's trying to preserve — an unavoidable, well-understood limitation rather than a flaw in method.

```
$ winpmem_mini.exe --output E:\memdump.raw
$ sha256sum memdump.raw > memdump.raw.sha256
```

Hashing the capture immediately afterward (Section 4.3's principle, applied here too) creates a verifiable fingerprint proving the file hasn't been altered since acquisition.

3.3 ANALYZING A MEMORY CAPTURE

Volatility is the standard open-source framework for analyzing memory captures — it can enumerate running processes at the time of capture (even ones since terminated), extract network connections, recover encryption keys, and detect certain rootkit and process-hiding techniques invisible to a live system's own task manager.

VOLATILITY PLUGIN	REVEALS
pslist / pstree	Running processes and their parent-child relationships — the memory-forensics equivalent of the Linux `ps --forest` triage step covered in the Linux for SOC Analysts guide, Section 6.1
netscan	Network connections active at the moment of capture
malfind	Memory regions showing signs of injected or hidden malicious code
dumpfiles	Extracts files that were open in memory at capture time

> **Memory capture should happen before disk imaging, per Section 2's order of volatility.** Once a suspect system's RAM is gone, it's gone permanently — there's no way to retroactively recover it from a disk image taken afterward. This is the single most time-sensitive step in the entire acquisition process.

// SECTION 4

Disk Forensics & Imaging

4.1 A BIT-FOR-BIT COPY, NOT A FILE COPY

A forensic **disk image** is a complete, bit-for-bit copy of every sector on a drive — including deleted files not yet overwritten, file system metadata, and unallocated space — fundamentally different from an ordinary file copy, which only captures files currently visible in the file system and none of that additional forensic detail.

4.2 WRITE-BLOCKING: NEVER TOUCH THE ORIGINAL

A **write blocker** — hardware or software — sits between the analyst's workstation and the original evidence drive, physically or logically preventing any write operation from reaching it, no matter what the analysis tooling attempts. This is Section 1.2's "never modify the original" principle made concrete: even accidentally mounting a drive read-write can alter timestamps and other metadata, potentially compromising its evidentiary value.

4.3 HASH BEFORE, HASH AFTER

```
$ dd if=/dev/sdb of=evidence.img bs=4M conv=sync,noerror
$ sha256sum /dev/sdb
$ sha256sum evidence.img
```

Hashing the original drive before imaging, and the resulting image file after, produces two hashes that should match exactly — cryptographic proof the image is a perfect, unaltered copy of the source. This hash pair becomes part of the chain of custody documentation (Section 5) and is often re-verified at multiple points throughout an investigation to prove the evidence was never tampered with.

4.4 WHAT ANALYSIS ACTUALLY LOOKS AT

ARTIFACT TYPE	REVEALS
File system metadata	Creation/modification/access timestamps — raw material for Section 6's timeline
Deleted files	Recoverable until overwritten — often significant evidence an attacker assumed was gone
Unallocated space	Fragments of previously deleted data, sometimes partially recoverable
Registry hives (Windows)	Detailed system and user activity history (Section 7)

> **All analysis happens on the image, never the original.** Once acquisition is complete and verified (Section 4.3), the original drive is set aside and secured (Section 5) — every subsequent analysis step in this guide works exclusively from the verified copy, so a mistake during analysis never risks the source evidence itself.

// SECTION 5

Chain of Custody

5.1 PROVABLE HANDLING, END TO END

Chain of custody is the documented, unbroken record of who possessed a piece of evidence, when, and what they did with it — from the moment it was collected until it's presented as a final finding. A gap in this record, however innocent, gives grounds to question whether evidence was tampered with in between, potentially undermining an otherwise sound investigation entirely.

5.2 WHAT A CHAIN OF CUSTODY RECORD ACTUALLY CONTAINS

FIELD	PURPOSE
Description of evidence	Exactly what was collected (a specific drive's serial number, a memory dump's hash)
Collection details	Who collected it, exactly when, and from where
Every transfer	Who had it next, when, and why — with no gaps in the timeline
Storage conditions	Where and how it was secured between each transfer
Hash values	The Section 4.3 hashes, re-verified and re-recorded at each transfer point

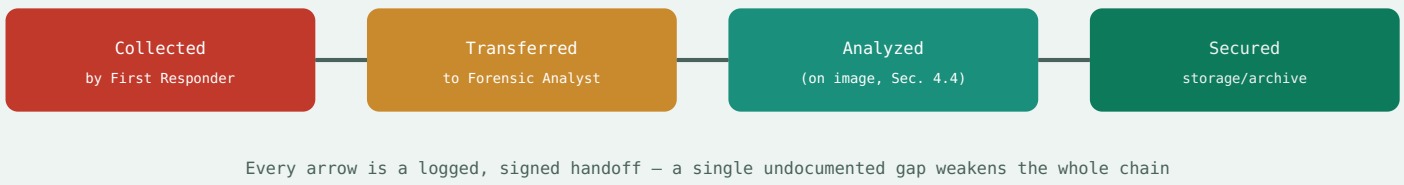

Fig 5.1 — Chain of custody: each handoff is individually documented, forming an unbroken, verifiable record from collection to final storage.

5.3 WHY THIS MATTERS EVEN OUTSIDE A COURTROOM

Not every investigation ends up in legal proceedings, but maintaining chain of custody discipline regardless is still worthwhile — it's the same rigor that makes an internal investigation's conclusions trustworthy to leadership, and it costs little to maintain from the start versus a great deal to reconstruct retroactively if a case unexpectedly does become legally significant later.

// SECTION 6

Timeline Analysis

6.1 RECONSTRUCTING "WHAT HAPPENED, IN ORDER"

Timeline analysis assembles timestamped events from every available source — file system metadata (Section 4.4), Windows Event Logs (Windows Event Log Reference guide), memory artifacts (Section 3.3) — into a single, chronologically ordered view of an incident, exactly the technique demonstrated at smaller scale in that guide's Section 8 worked example, now applied with the fuller evidence set a forensic investigation provides.

6.2 THE MACB TIMESTAMPS

TIMESTAMP	MEANING
Modified	When a file's content was last changed
Accessed	When a file was last read/opened
Changed	When a file's metadata (permissions, etc.) last changed
Birth (created)	When a file was originally created

These four timestamps, extracted from file system metadata, form the backbone of most timeline analysis — a file's Birth and Modified times close together but its Accessed time much later, for instance, can indicate exactly when a dormant, previously-dropped file was finally executed.

6.3 TIMESTAMPS CAN BE MANIPULATED — CORROBORATE WHERE POSSIBLE

> **An attacker with sufficient access can alter file timestamps directly ("timestomping") to blend malicious files in with legitimate system activity.** This is precisely why a timeline built from a *single* source is weaker evidence than one corroborated across multiple independent sources — a file's claimed creation timestamp is far more trustworthy when it lines up with an independent Windows Event Log entry (Windows Event Log Reference guide, Section 4.1's Event ID 4688) or a memory-captured process start time (Section 3.3) that would be much harder for an attacker to alter consistently across all three.

6.4 TOOLING

Tools like `log2timeline/plaso` automate pulling timestamped events from dozens of different artifact types across a disk image into one unified timeline, which analysts then filter and correlate manually — turning what would be an enormous manual cross-referencing task into a structured, searchable dataset.

// SECTION 7

Windows Forensic Artifacts

7.1 BEYOND THE EVENT LOG

The Windows Event Log Reference guide covers the primary auditing channel, but Windows leaves forensically valuable traces in many other places too — often surviving even when logging itself was disabled or an attacker cleared the Event Log entirely (a technique that, notably, is itself logged and highly suspicious on its own).

ARTIFACT	REVEALS
Prefetch files	Evidence a specific executable ran, including run count and last-run time — persists even if the executable itself was later deleted
Registry — Run keys	Persistence mechanisms configured to launch at startup (echoing the persistence checklist in the Linux for SOC Analysts guide, Section 7.2, and the service/task Event IDs in the Windows Event Log Reference guide, Section 5)
Registry — UserAssist	A record of GUI applications a user has launched, with timestamps
Shellbags	Evidence of folders a user browsed, even ones since deleted
$MFT (Master File Table)	NTFS's own file record — a rich source of the MACB timestamps from Section 6.2, and evidence of deleted files
Windows Event Log	Full coverage in the Windows Event Log Reference guide

7.2 WHY PREFETCH IS ESPECIALLY VALUABLE

Prefetch files are created by Windows itself, for performance reasons entirely unrelated to security — meaning most attackers don't think to clean them up the way they might think to clear the Event Log. A Prefetch entry for an executable that no longer exists anywhere on disk is strong evidence that executable ran at some point and was later deleted, directly corroborating (Section 6.3's principle) other evidence of its execution.

7.3 REGISTRY AS A FORENSIC GOLDMINE

Beyond persistence mechanisms, the Windows Registry — accessible for offline analysis directly from a disk image's hive files, without needing to boot the original system — records an enormous amount of user and system activity history: recently opened files, connected USB devices, installed software, and network configuration history, much of which persists long after the activity itself.

> **Cross-referencing artifacts is where real findings emerge.** Any single artifact in this section, alone, is suggestive at best. A Prefetch entry, a Run key pointing at the same executable, and a corroborating memory-captured process (Section 3.3) — all pointing at the same file, the same rough timeframe — is the kind of multi-source confirmation (Section 6.3) that turns a hypothesis into a defensible finding.

// SECTION 8

Worked Example: A Forensic Investigation

Following an IR engagement's containment phase (IR Playbook Companion guide, Section 4), a compromised Windows workstation is handed off for full forensic investigation. Walking the process built across this guide:

1. **Acquisition, in order of volatility (Section 2):** memory captured first (Section 3) via WinPmem, then the disk imaged (Section 4) with a hardware write blocker, both hashed immediately per Section 4.3.
2. **Chain of custody established (Section 5):** both the memory dump and disk image are logged, hashed, and transferred to the forensic analyst with a signed handoff record.
3. **Memory analysis (Section 3.3):** Volatility's `pslist` reveals a process with no corresponding legitimate application, and `netscan` shows it held an active connection to an external IP at capture time.
4. **Disk analysis — Prefetch & Registry (Section 7):** a Prefetch entry confirms the same executable ran three times over the past week; a Run key under `HKCU` configured it to launch at every logon — the persistence mechanism.
5. **Timeline construction (Section 6):** the executable's Birth timestamp, its first Prefetch run, and a corroborating Windows Event ID 4688 (Windows Event Log Reference guide, Section 4.1) all align within the same few minutes — high-confidence, multi-source corroboration per Section 6.3.
6. **Scope check:** the $MFT (Section 7.1) reveals two additional files created moments after the initial executable — a dropped secondary payload, previously unknown to the investigation.

> **Final finding, fully corroborated:** initial execution confirmed by three independent sources (Prefetch, Registry, Event Log), persistence mechanism identified, and a previously-unknown secondary payload discovered purely through timeline analysis — a defensible, evidence-backed conclusion built entirely from artifacts that would have been destroyed by simply powering off the machine and skipping Section 2's order of volatility.

// SECTION 9

Quick Reference & Glossary

TERM	DEFINITION
Digital Forensics	The scientific, defensible process of identifying, preserving, and analyzing digital evidence.
Order of Volatility	The standard evidence collection sequence, most volatile (RAM) to least (archives).
Write Blocker	Hardware/software preventing any write operation from reaching original evidence.
Disk Image	A complete, bit-for-bit copy of a drive, including deleted files and unallocated space.
Chain of Custody	The documented, unbroken record of who handled evidence, when, and why.
MACB Timestamps	Modified, Accessed, Changed, Birth — the four core file system timestamps.
Timestomping	An anti-forensic technique that alters file timestamps to hide malicious activity.
Prefetch	A Windows-created record of executed applications, including run count and last-run time.
Shellbags	Windows Registry artifacts recording folders a user has browsed.
$MFT	Master File Table — NTFS's own file record, a rich source of file metadata and deleted-file evidence.
Volatility	An open-source framework for analyzing memory captures.

End of guide. Every technique here traces back to Section 1's three principles — preserve the original, document everything, and make it reproducible. The specific tools and artifact locations will keep changing; that discipline is what actually makes an investigation's conclusions worth trusting.